Pangolin Wildlife Trade Spatial Modeling - Gabon

Objective 1: Expert and community members annotation difference significance maps

Elda Yitbarek Bezuayene

2026-08-15

```
# Load required packages
library(sf)
```

```
## Linking to GEOS 3.14.1, GDAL 3.12.1, PROJ 9.7.1; sf_use_s2() is TRUE
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(spatstat)
```

```
## Loading required package: spatstat.data
```

```
## Loading required package: spatstat.univar
```

```
## spatstat.univar 3.1-7
```

```
## Loading required package: spatstat.geom
```

```
## spatstat.geom 3.7-3
```

```
## Loading required package: spatstat.random
```

```
## spatstat.random 3.4-5
```

```
## Loading required package: spatstat.explore
```

```
## Loading required package: nlme

##
## Attaching package: 'nlme'

## The following object is masked from 'package:dplyr':
##
##     collapse

## spatstat.explore 3.8-0

## Loading required package: spatstat.model

## Loading required package: rpart

## spatstat.model 3.7-0

## Loading required package: spatstat.linnet

## spatstat.linnet 3.5-0

##
## spatstat 3.6-0
## For an introduction to spatstat, type 'beginner'
```

```
library(spatstat.geom)
library(spatstat.explore)
library(spatstat.random)
library(stars)
```

```
## Loading required package: abind
```

```
library(terra)
```

```
## terra 1.9.11

##
## Attaching package: 'terra'

## The following objects are masked from 'package:spatstat.geom':
##
##     area, delaunay, is.empty, rescale, rotate, shift, where.max,
##     where.min
```

```
library(tmap)
```

KDE of the three crime stages

Importing and preparation

```r
# ==============================================================================
# CHUNK 1: SETUP, ENVIRONMENT, PATHS, AND HELPER FUNCTIONS
# ==============================================================================

# Load required packages
library(sf)
library(spatstat)
library(spatstat.geom)
library(spatstat.explore)
library(spatstat.random)
library(stars)
library(terra)
library(tmap)

# ------------------------------------------------------------------------------
# 1. DIRECTORY PATHS & GLOBAL PARAMETERS
# ------------------------------------------------------------------------------
# Set base directory and relative sub-paths
base_dir   <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_
input_dir  <- file.path(base_dir, "Input/Objective_1")
aux_dir    <- file.path(base_dir, "AuxiliaryDatasets")
output_dir <- file.path(base_dir, "Output/Objective_1")

# Define spatial parameters
target_crs   <- 5223  # Projected CRS for Gabon (Meters)
cell_size_km <- 2     # Grid resolution: 2 km x 2 km

# Define permutation test parameters
alpha  <- 0.05
n_perm <- 999
stages <- c("I", "II", "III")

set.seed(42)  # For reproducible permutation sampling

# ------------------------------------------------------------------------------
# 2. HELPER FUNCTIONS
# ------------------------------------------------------------------------------

#' Clean Point Geometries
#' Removes empty, NA, invalid, or duplicate spatial point records.
clean_points <- function(gdf) {
  gdf <- gdf[!st_is_empty(gdf), ]
  gdf <- gdf[!is.na(st_geometry(gdf)), ]
  gdf <- gdf[st_is_valid(gdf), ]
  gdf <- gdf[!duplicated(st_geometry(gdf)), ]
  return(gdf)
}

#' Convert Simple Features (sf) Points to spatstat ppp in Kilometers
make_ppp_from_sf <- function(gdf, window_m, window_km) {
  coords <- st_coordinates(gdf)
  coords <- coords[complete.cases(coords[, 1:2]), ]
```

```
  # Construct initial ppp in meters
  ppp_m <- ppp(x = coords[, 1], y = coords[, 2], window = window_m, check = TRUE)

  # Rescale to kilometers and attach master kilometer observation window
  ppp_km <- spatstat.geom::rescale(ppp_m, s = 1000, unitname = "km")
  Window(ppp_km) <- window_km

  return(ppp_km)
}

# ------------------------------------------------------------------------------
# 3. IMPORT & PREPARE OBSERVATION WINDOW
# ------------------------------------------------------------------------------
# Load 15 km buffered national boundary of Gabon
gabon_window_sf <- st_read(
  file.path(aux_dir, "Gabon_country_boundary_UNOCHA_EPSG5223_15kmBuffer.gpkg"),
  quiet = TRUE
) %>%
  st_make_valid() %>%
  st_transform(target_crs) %>%
  st_union()

# Convert boundary to spatstat observation windows (Meters and Kilometers)
gabon_window_m  <- as.owin(gabon_window_sf)
gabon_window_km <- spatstat.geom::rescale(gabon_window_m, s = 1000, unitname = "km")
```

```
# ==============================================================================
# CHUNK 2: CRIME STAGES IMPORT & DYNAMIC BANDWIDTH KDE
# ==============================================================================

# 1. Import and clean raw spatial point data for all three crime stages
stage_1_sf <- clean_points(st_read(file.path(input_dir, "Crime_stage_I.gpkg"), quiet = TRUE))
stage_2_sf <- clean_points(st_read(file.path(input_dir, "Crime_stage_II.gpkg"), quiet = TRUE))
stage_3_sf <- clean_points(st_read(file.path(input_dir, "Crime_stage_III.gpkg"), quiet = TRUE))

# 2. Convert sf layers to spatstat point patterns (ppp) in kilometers
stage_1_ppp <- make_ppp_from_sf(stage_1_sf, gabon_window_m, gabon_window_km)
stage_2_ppp <- make_ppp_from_sf(stage_2_sf, gabon_window_m, gabon_window_km)
stage_3_ppp <- make_ppp_from_sf(stage_3_sf, gabon_window_m, gabon_window_km)

# 3. Calculate optimal bandwidths using Likelihood Cross-Validation (bw.ppl)
sigma_1 <- bw.ppl(stage_1_ppp)
sigma_2 <- bw.ppl(stage_2_ppp)
sigma_3 <- bw.ppl(stage_3_ppp)

# Compute mean bandwidth across all stages to standardize spatial smoothing
mean_sigma <- mean(c(sigma_1, sigma_2, sigma_3))

cat(sprintf("Selected Likelihood Bandwidths -> Stage I: %.2f km | Stage II: %.2f km | Stage III: %.2f km
            sigma_1, sigma_2, sigma_3))
```

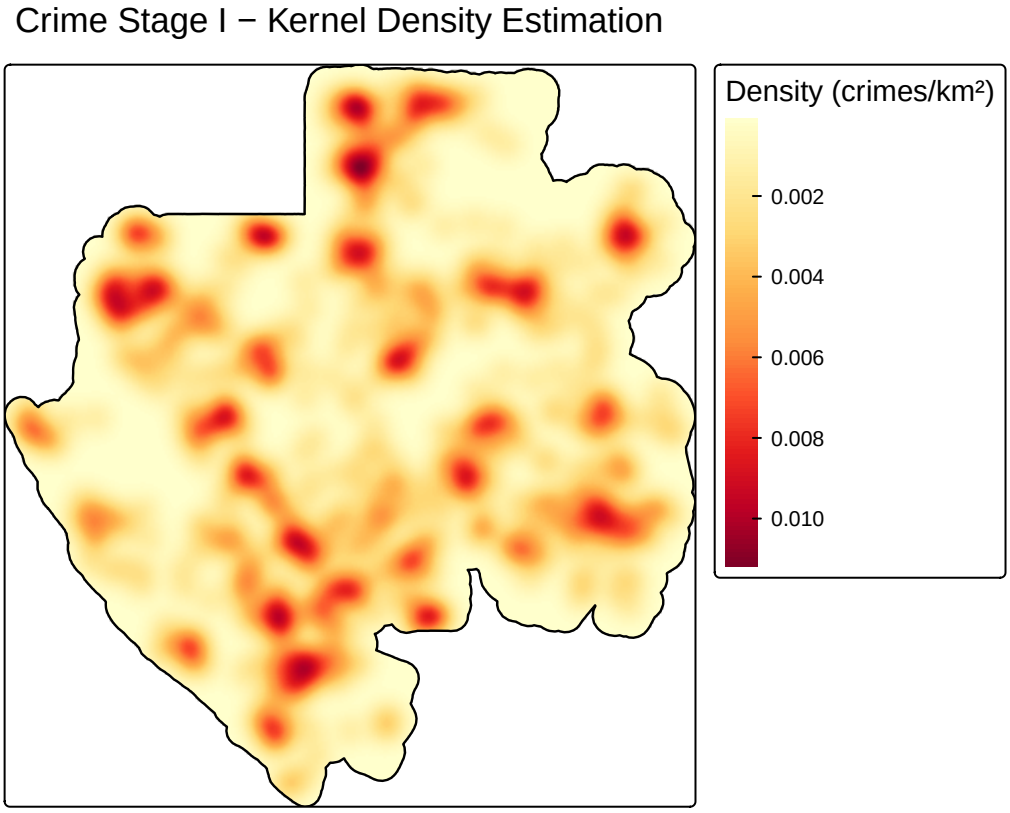
```
## Selected Likelihood Bandwidths -> Stage I: 12.31 km | Stage II: 11.08 km | Stage III: 12.55 km
```

```r
cat(sprintf("Mean Bandwidth Applied Across All KDE Computations: %.2f km\n", mean_sigma))
```

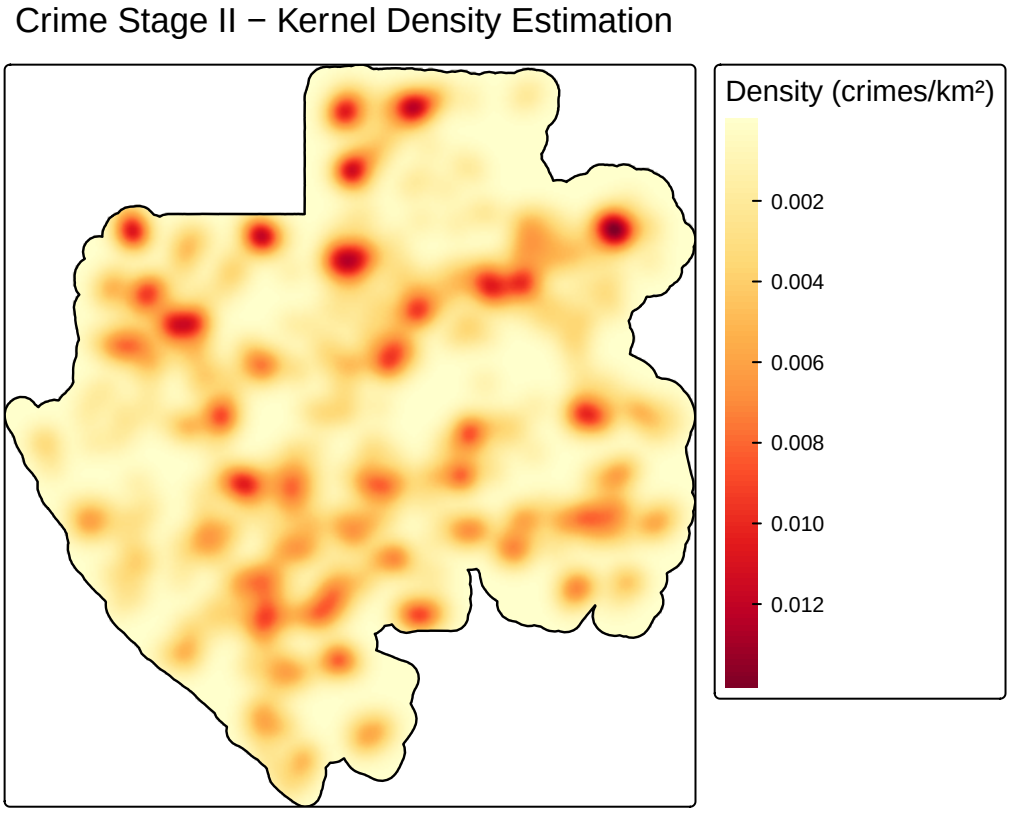
```
## Mean Bandwidth Applied Across All KDE Computations: 11.98 km
```

```r
# 4. Generate Kernel Density Estimation (KDE) pixel images (crimes/km2)
kde_stage_1 <- density.ppp(stage_1_ppp, sigma = mean_sigma, eps = cell_size_km, edge = TRUE, diggle = TR
kde_stage_2 <- density.ppp(stage_2_ppp, sigma = mean_sigma, eps = cell_size_km, edge = TRUE, diggle = TR
kde_stage_3 <- density.ppp(stage_3_ppp, sigma = mean_sigma, eps = cell_size_km, edge = TRUE, diggle = TR
```

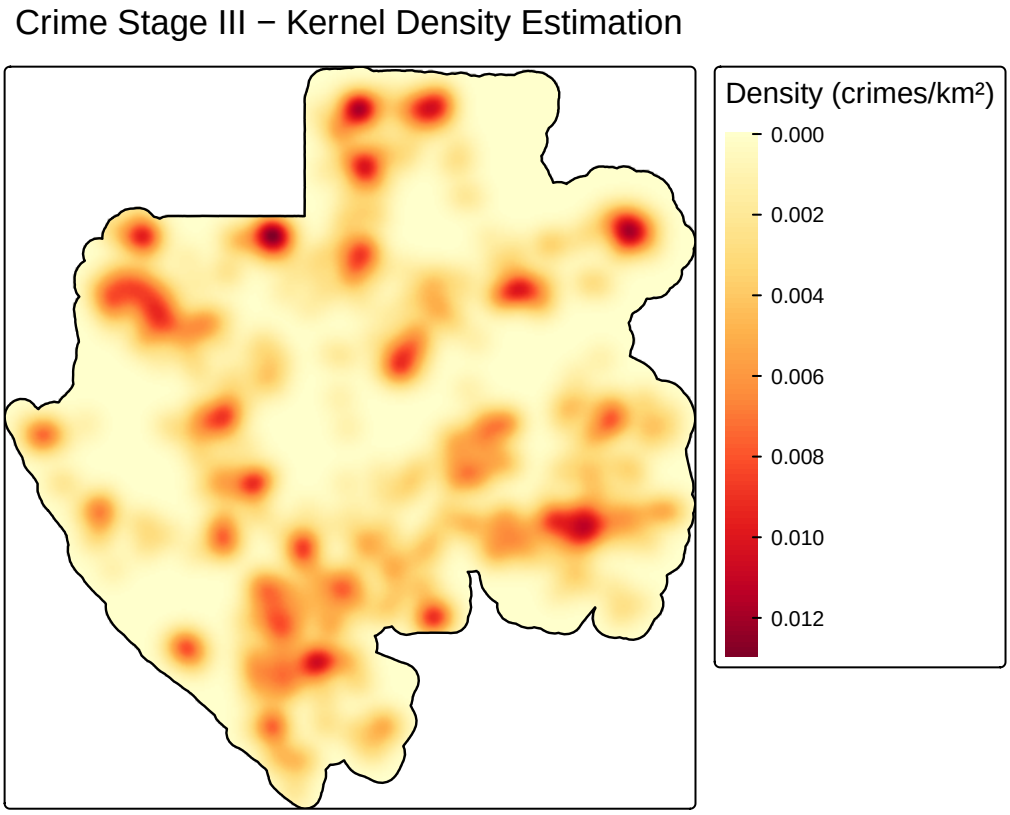
```r
# ==============================================================================
# CHUNK 3: RASTER CONVERSION & MAP VISUALIZATION
# ==============================================================================

#' Helper Function: Convert spatstat 'im' to 'stars' object in meters (EPSG:5223)
im_to_stars <- function(kde_im, mask_sf, crs = 5223) {
  kde_m <- spatstat.geom::rescale(kde_im, 1/1000, "m")
  rast  <- st_set_crs(stars::st_as_stars(kde_m), crs)
  return(rast[mask_sf])  # Clip raster extent to Gabon boundary window
}

# Convert KDE images to spatial raster objects
kde_1_rast <- im_to_stars(kde_stage_1, gabon_window_sf, target_crs)
kde_2_rast <- im_to_stars(kde_stage_2, gabon_window_sf, target_crs)
kde_3_rast <- im_to_stars(kde_stage_3, gabon_window_sf, target_crs)

# Set tmap mode to static plotting
tmap_mode("plot")

#' Helper Function: Render Standardized tmap for KDE Rasters
plot_kde <- function(rast_obj, title_text) {
  tm_shape(rast_obj) +
    tm_raster(title = "Density (crimes/km²)", palette = "YlOrRd", style = "cont") +
    tm_shape(gabon_window_sf) +
    tm_borders(col = "black", lwd = 1.2) +
    tm_layout(main.title = title_text, main.title.size = 1.1)
}

# Display maps
plot_kde(kde_1_rast, "Crime Stage I - Kernel Density Estimation")
```




```
plot_kde(kde_2_rast, "Crime Stage II - Kernel Density Estimation")
```




```
plot_kde(kde_3_rast, "Crime Stage III - Kernel Density Estimation")
```



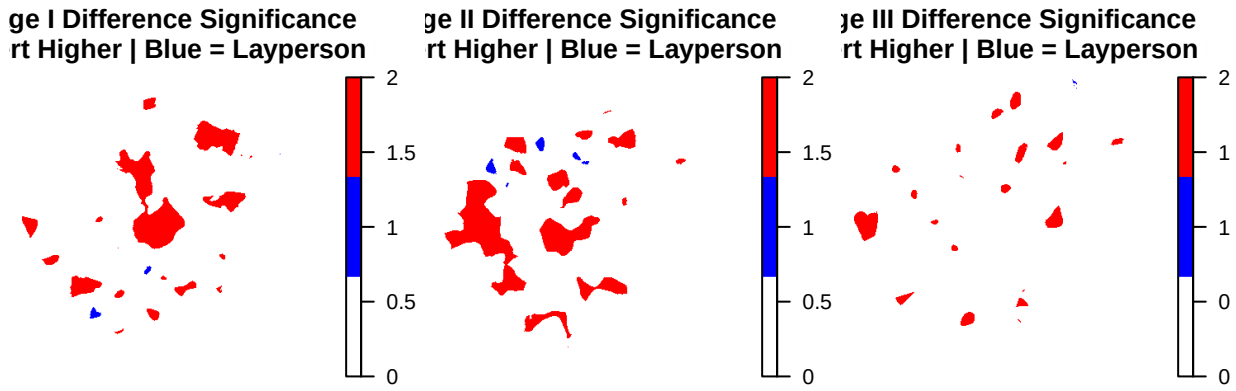

```
# ============================================================================
# CHUNK 4: PARALLELIZED PERMUTATION TEST (EXPERT VS. LAYPERSON) & EXPORT
# ============================================================================

library(future)
library(future.apply)
library(spatstat)
library(sf)
library(terra)

# Set up multi-core parallel backend (uses all cores minus 1)
plan(multisession, workers = availableCores() - 1)

#' Calculate Normalized KDE (Integrates to 1 over observation window)
get_kde_normalized <- function(ppp_obj, sigma_val, eps_km = cell_size_km) {
  n <- ppp_obj$n
  if (n == 0) stop("Point pattern contains zero points.")

  kde <- density.ppp(
    ppp_obj,
    sigma   = sigma_val,
    eps     = eps_km,
    weights = rep(1 / n, n),
    edge    = TRUE,
    diggle  = TRUE
  )
```

```r
  return(kde / integral.im(kde))
}

#' Two-Sided Parallelized Permutation Significance Test
permutation_pvalue_test_parallel <- function(ppp_exp, ppp_non, sigma_val, n_perm = 999) {
  n_exp <- ppp_exp$n
  n_non <- ppp_non$n

  all_coords <- rbind(cbind(ppp_exp$x, ppp_exp$y), cbind(ppp_non$x, ppp_non$y))
  n_total    <- nrow(all_coords)
  win        <- ppp_exp$window

  # Compute observed normalized KDE difference (Expert minus Layperson)
  kde_exp_obs <- get_kde_normalized(ppp_exp, sigma_val = sigma_val, eps_km = cell_size_km)
  kde_non_obs <- get_kde_normalized(ppp_non, sigma_val = sigma_val, eps_km = cell_size_km)

  observed_diff <- eval.im(kde_exp_obs - kde_non_obs)
  obs_vals_abs  <- abs(observed_diff$v)

  # Execute Monte Carlo Permutations in Parallel
  perm_counts_list <- future_lapply(1:n_perm, function(p) {
    perm_idx <- sample.int(n_total, size = n_total, replace = FALSE)

    ppp_exp_perm <- ppp(
      all_coords[perm_idx[1:n_exp], 1],
      all_coords[perm_idx[1:n_exp], 2],
      window = win, check = FALSE
    )
    ppp_non_perm <- ppp(
      all_coords[perm_idx[(n_exp + 1):n_total], 1],
      all_coords[perm_idx[(n_exp + 1):n_total], 2],
      window = win, check = FALSE
    )

    kde_exp_perm <- get_kde_normalized(ppp_exp_perm, sigma_val = sigma_val, eps_km = cell_size_km)
    kde_non_perm <- get_kde_normalized(ppp_non_perm, sigma_val = sigma_val, eps_km = cell_size_km)

    perm_diff <- eval.im(kde_exp_perm - kde_non_perm)

    # Return binary boolean matrix of exceedances
    return(as.numeric(abs(perm_diff$v) >= obs_vals_abs))
  }, future.seed = TRUE)

  # Reduce parallel outputs into a single sum matrix
  total_exceedance <- Reduce("+", perm_counts_list)

  # Compute exact Monte Carlo p-values
  p_values   <- observed_diff
  p_values$v <- (total_exceedance + 1) / (n_perm + 1)

  return(list(observed_diff = observed_diff, p_values = p_values))
}
```

```r
# Setup plotting grid for loop outputs
par(mfrow = c(1, 3), mar = c(1, 1, 3, 1))

results_list <- list()

# Execute significance testing across all crime stages
for (stage in stages) {
  cat("\n============================================================\n")
  cat("Running Parallel Significance Test for Crime Stage", stage, "\n")
  cat("============================================================\n")

  # Import expert and non-expert layers
  exp_sf <- clean_points(st_read(file.path(output_dir, paste0("CrimeStage_", stage, "_Expert.gpkg")), qu
    st_transform(target_crs)
  non_sf <- clean_points(st_read(file.path(output_dir, paste0("CrimeStage_", stage, "_nonExpert.gpkg"))
    st_transform(target_crs)

  # Convert to kilometer ppp
  ppp_exp <- make_ppp_from_sf(exp_sf, gabon_window_m, gabon_window_km)
  ppp_non <- make_ppp_from_sf(non_sf, gabon_window_m, gabon_window_km)

  # Run parallel permutation test
  res <- permutation_pvalue_test_parallel(
    ppp_exp   = ppp_exp,
    ppp_non   = ppp_non,
    sigma_val = mean_sigma,
    n_perm    = n_perm
  )

  # Create categorical significance raster map
  # 0 = Not Significant | 1 = Layperson Significantly Higher | 2 = Expert Significantly Higher
  sig_map <- res$observed_diff
  sig_map$v[] <- 0
  sig_map$v[(res$observed_diff$v > 0) & (res$p_values$v < alpha)] <- 2
  sig_map$v[(res$observed_diff$v < 0) & (res$p_values$v < alpha)] <- 1

  # Convert spatial coordinates back to meters before export
  sig_map_m <- spatstat.geom::rescale(sig_map, s = 1 / 1000, unitname = "m")
  sig_rast  <- terra::rast(sig_map_m)
  terra::crs(sig_rast) <- paste0("EPSG:", target_crs)

  # Export to GeoPackage
  stage_num   <- match(stage, stages)
  output_path <- file.path(output_dir, paste0("significance_test_crime_stage_", stage_num, "v_100.gpkg"

  terra::writeRaster(
    sig_rast,
    filename  = output_path,
    filetype  = "GPKG",
    overwrite = TRUE,
    gdal      = c("RASTER_TABLE=significance")
  )
```

```r
  # Render base plot output
  plot(
    sig_map,
    col     = c("white", "blue", "red"),
    main    = paste0("Stage ", stage, " Difference Significance\nRed = Expert Higher | Blue = Layperson
    ribargs = list(las = 1)
  )

  results_list[[stage]] <- sig_map
}
```

```
##
## ============================================================
## Running Parallel Significance Test for Crime Stage I
## ============================================================

##
## ============================================================
## Running Parallel Significance Test for Crime Stage II
## ============================================================

##
## ============================================================
## Running Parallel Significance Test for Crime Stage III
## ============================================================
```



```r
# Reset parallel plan back to sequential after run
plan(sequential)
```

Summary Statistics Computing area of community members and expert annotation

```r
# ============================================================================
# COMPUTE AREA PERCENTAGE FROM EXPORTED SIGNIFICANCE GPKG RASTERS
# ============================================================================

library(terra)

# ----------------------------------------------------------------------------
# 1. Define paths and class labels
# ----------------------------------------------------------------------------

output_dir <- "C:/Users/eldayit/OneDrive - University of Maryland/Desktop/All files/Ventures/Pangolins_C

sig_files <- list(
  "Stage I"   = file.path(output_dir, "significance_test_crime_stage_1.gpkg"),
  "Stage II"  = file.path(output_dir, "significance_test_crime_stage_2.gpkg"),
  "Stage III" = file.path(output_dir, "significance_test_crime_stage_3.gpkg")
)

class_labels <- data.frame(
  class_value = c(0, 1, 2),
  class_label = c(
    "Not significant",
    "Layperson significantly higher",
    "Expert significantly higher"
  )
)

# ----------------------------------------------------------------------------
# 2. Function to compute area percentage (Quiet version)
# ----------------------------------------------------------------------------

compute_sig_area_stats <- function(raster_path, stage_name) {

  # Read GPKG raster
  r <- terra::rast(raster_path)

  # Pixel area in km2 (convert m2 to km2 for EPSG:5223)
  pixel_area_km2 <- prod(terra::res(r)) / 1e6

  # Extract raster values & remove NA
  vals <- terra::values(r, mat = FALSE)
  vals <- vals[!is.na(vals)]
  vals <- as.integer(round(vals))

  # Count classes 0, 1, 2
  class_counts <- table(factor(vals, levels = c(0, 1, 2)))

  total_pixels <- sum(class_counts)
```

```r
  stats <- data.frame(
    crime_stage  = stage_name,
    class_value  = c(0, 1, 2),
    class_label  = class_labels$class_label,
    pixel_count  = as.integer(class_counts),
    area_km2     = as.integer(class_counts) * pixel_area_km2,
    percent_area = (as.integer(class_counts) / total_pixels) * 100
  )

  # Print only the individual Area Statistics table
  cat("\n============================================================\n")
  cat("Area Statistics:", stage_name, "\n")
  cat("============================================================\n")
  print(stats)

  return(stats)
}

# ------------------------------------------------------------------------------
# 3. Run for all three crime stages and display combined summary
# ------------------------------------------------------------------------------

all_stats <- list()

for (stage_name in names(sig_files)) {
  all_stats[[stage_name]] <- compute_sig_area_stats(
    raster_path = sig_files[[stage_name]],
    stage_name  = stage_name
  )
}
```

```
##
## ============================================================
## Area Statistics: Stage I
## ============================================================
##   crime_stage class_value                    class_label pixel_count
## 1     Stage I           0                Not significant      115555
## 2     Stage I           1 Layperson significantly higher          89
## 3     Stage I           2    Expert significantly higher        7752
##      area_km2 percent_area
## 1 460125.8725  93.64566112
## 2    354.3871   0.07212551
## 3  30867.5156   6.28221336
##
## ============================================================
## Area Statistics: Stage II
## ============================================================
##   crime_stage class_value                    class_label pixel_count   area_km2
## 1    Stage II           0                Not significant      112948 449745.117
## 2    Stage II           1 Layperson significantly higher         306   1218.455
## 3    Stage II           2    Expert significantly higher       10142  40384.203
##   percent_area
## 1   91.5329508
```

```
## 2    0.2479821
## 3    8.2190671
##
## ============================================================
## Area Statistics: Stage III
## ============================================================
##   crime_stage class_value                   class_label pixel_count
## 1   Stage III           0               Not significant      120503
## 2   Stage III           1 Layperson significantly higher           6
## 3   Stage III           2    Expert significantly higher        2887
##       area_km2 percent_area
## 1 479828.20313 97.655515576
## 2     23.89127  0.004862394
## 3  11495.68079  2.339622030
```

```
combined_stats <- do.call(rbind, all_stats)

cat("\n\n============================================================\n")
```

```
##
##
## ============================================================
```

```
cat("COMBINED SIGNIFICANCE AREA SUMMARY\n")
```

```
## COMBINED SIGNIFICANCE AREA SUMMARY
```

```
cat("============================================================\n")
```

```
## ============================================================
```

```
print(combined_stats, row.names = FALSE)
```

```
##  crime_stage class_value                   class_label pixel_count
##      Stage I           0               Not significant      115555
##      Stage I           1 Layperson significantly higher          89
##      Stage I           2    Expert significantly higher        7752
##     Stage II           0               Not significant      112948
##     Stage II           1 Layperson significantly higher         306
##     Stage II           2    Expert significantly higher       10142
##    Stage III           0               Not significant      120503
##    Stage III           1 Layperson significantly higher           6
##    Stage III           2    Expert significantly higher        2887
##      area_km2 percent_area
##  460125.87249 93.645661124
##     354.38711  0.072125515
##   30867.51559  6.282213362
##  449745.11744 91.532950825
##    1218.45456  0.247982106
##   40384.20318  8.219067069
##  479828.20313 97.655515576
##      23.89127  0.004862394
##   11495.68079  2.339622030
```